Python Basics · Day 5 of 30 · about 60 minutes

Repeating work with loops

🎯 Today's goal

By the end of this hour you will:

- repeat code for every item in a list with a **`for` loop**
- add things up and count things inside a loop
- build a new list, step by step, with `.append()`
- use `range()`, `enumerate()` and loop over a dictionary
- repeat while something is true with a **`while` loop**, and stop early with `break`

🤔 Why this matters

Imagine checking a list of 10 000 temperatures for fevers by writing 10 000 `if` lines. Nobody does that. Instead you write the check **once** and tell Python: "do this for every value in the list." That's a **loop**.

It's like a cashier scanning a basket: pick up an item, scan it, add its price to the total, pick up the next item… until the basket is empty. Computers are brilliant at this kind of boring repetition, and almost every piece of data analysis is a loop underneath, even when a library like pandas hides it from you.

🔁 The `for` loop

Here are a (made-up) café's sales, in dollars, for one week. A `for` loop takes the items **one at a time**, puts each one in a variable (here called `amount`), and runs the indented block for it:

```
sales = [120, 95, 140, 80, 160, 210, 175]

for amount in sales:
    print("Sold for", amount, "dollars")

print("Done!")     # not indented: runs once, after the loop
```

```
Sold for 120 dollars
Sold for 95 dollars
Sold for 140 dollars
Sold for 80 dollars
Sold for 160 dollars
Sold for 210 dollars
Sold for 175 dollars
Done!
```

Read it as: "**for** each `amount` **in** the list `sales`, print it." The pattern is the same as an `if`: a colon at the end of the line, and a block indented by 4 spaces. Each trip through the block is called an **iteration**. The name `amount` is your choice; pick one that describes a single item (`for temp in temps`, `for name in names`).

➕ Adding up: the accumulator

The most useful loop pattern: start a variable at 0 **before** the loop, and add to it **inside** the loop. The variable is called an **accumulator** (it accumulates, or collects, the result):

```
total = 0                  # 1. start at zero, before the loop

for amount in sales:
    total += amount        # 2. add each amount inside the loop
    print(f"after adding {amount}: total is {total}")

print("Week total:", total)
print("Check with sum():", sum(sales))
```

```
after adding 120: total is 120
after adding 95: total is 215
after adding 140: total is 355
after adding 80: total is 435
after adding 160: total is 595
after adding 210: total is 805
after adding 175: total is 980
Week total: 980
Check with sum(): 980
```

Yes, `sum()` does this in one go. But `sum()` can only add. With a loop you can do *anything* for each item, like counting only some of them. Put an `if` inside the loop, and add 1 to a **counter** each time the condition is true:

```
busy_days = 0

for amount in sales:
    if amount > 150:
        busy_days += 1     # one more busy day

print(f"{busy_days} of {len(sales)} days sold more than 150 dollars")
```

```
3 of 7 days sold more than 150 dollars
```

Watch the indentation. `busy_days += 1` is pushed in 8 spaces: it belongs to the `if`, which belongs to the `for`. The final `print` has no spaces, so it runs once, after the loop has finished.

🧺 Building a new list

Another everyday pattern: start with an **empty list** `[]` and `.append()` something to it in every iteration. Here we convert made-up temperatures from Celsius to Fahrenheit, and label each day:

```
temps_c = [21.5, 33.0, 19.8, 30.1, 27.3]

temps_f = []
labels = []
for t in temps_c:
    temps_f.append(round(t * 9 / 5 + 32, 1))
    if t >= 30:
        labels.append("hot")
    else:
        labels.append("ok")

print(temps_f)
print(labels)
```

```
[70.7, 91.4, 67.6, 86.2, 81.1]
['ok', 'hot', 'ok', 'hot', 'ok']
```

🔢 Counting with `range()`

Sometimes you don't have a list; you just want to repeat something a number of times, or go through a series of numbers. `range()` makes that series. Like slicing, it stops **just before** the stop

number:

You write	You get
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 20, 5)`	0, 5, 10, 15 (the third number is the **step**)
`range(10, 0, -2)`	10, 8, 6, 4, 2 (a negative step counts down)

```
for week in range(1, 4):
    print("Week", week)

print(list(range(0, 20, 5)))     # list() shows all the numbers at once
print(list(range(10, 0, -2)))
```

```
Week 1
Week 2
Week 3
[0, 5, 10, 15]
[10, 8, 6, 4, 2]
```

A `for` loop needs something with items to go through. A plain number has no items, so this fails:

💡 This example has a mistake on purpose. Run it and read the error.

```
for i in 5:
    print(i)
```

⚠️ TypeError

You combined things that don't fit together, like text + a number.

```
Traceback (most recent call last):
  line 1, in <module>
    for i in 5:
             ^
TypeError: 'int' object is not iterable
```

'int' object is not iterable means "I can't go through a whole number item by item". (**Iterable** means "something a loop can go through", like a list, a string or a range.) The fix: `for i in range(5):`.

🏷️ Position numbers and dictionaries

Often you want the item **and** its position. `enumerate()` gives you both, as two variables at once. Add `start=1` to count from 1 like humans do:

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

for i, day in enumerate(days):
    print(day, "sold", sales[i])     # i is the index, so sales[i] is that day's sales

for number, day in enumerate(days, start=1):
    print(f"Day {number}: {day}")
```

```
Mon sold 120
Tue sold 95
Wed sold 140
Thu sold 80
Fri sold 160
Sat sold 210
Sun sold 175
Day 1: Mon
Day 2: Tue
Day 3: Wed
Day 4: Thu
Day 5: Fri
Day 6: Sat
Day 7: Sun
```

Looping over a dictionary works the same way with `.items()` from Day 3: each iteration gives you a key **and** its value:

```
stock = {"apple": 40, "banana": 6, "cherry": 0, "kiwi": 15}   # made-up kilos in stock

for fruit, kilos in stock.items():
    if kilos == 0:
        print(f"{fruit}: SOLD OUT")
    elif kilos < 10:
        print(f"{fruit}: only {kilos} kg left, order more")
    else:
        print(f"{fruit}: {kilos} kg")
```

```
apple: 40 kg
banana: only 6 kg left, order more
cherry: SOLD OUT
kiwi: 15 kg
```

⏳ The `while` loop

A `for` loop runs once per item. A `while` loop runs **as long as a condition is true**, like an `if` that keeps repeating. It's useful when you don't know in advance how many rounds you need. How many months does it take to save 1000 dollars, putting away 150 a month?

```
savings = 0
months = 0

while savings < 1000:
    savings += 150
    months += 1

print(f"After {months} months you have {savings} dollars")
```

```
After 7 months you have 1050 dollars
```

Before each round Python asks the question again: is `savings < 1000` still true? When the answer becomes `False`, the loop ends. Here is a countdown:

```
count = 5
while count > 0:
    print(count)
    count -= 1        # without this line the loop would never end!
print("Lift off!")
```

```
5
4
3
2
1
Lift off!
```

Endless loops. If nothing inside a `while` loop ever makes its condition `False`, it runs forever. On this page Python stops your code after **20 seconds** with a `TimeLimitExceeded` message (and the page freezes until then). If you see it, look for the line that should change the condition, like `count -= 1`.

🛑 Stopping early: `break` and `continue`

`break` leaves the loop **immediately**. It's perfect for "find the first one": once you've found it, there's no need to look further. `continue` skips the rest of **this** round and jumps to the next item.

```
# Find the first day the café sold more than 150 dollars
for i, amount in enumerate(sales):
    if amount > 150:
        print(f"First big day: {days[i]} with {amount} dollars")
        break
    print(f"{days[i]}: {amount}, not yet")
```

```
Mon: 120, not yet
Tue: 95, not yet
Wed: 140, not yet
Thu: 80, not yet
First big day: Fri with 160 dollars
```

```
# Some readings are missing (None). Skip them, average the rest.
readings = [36.6, None, 37.2, 36.9, None, 38.1]

total = 0
valid = 0
for r in readings:
    if r is None:
        continue          # skip this one, go to the next reading
    total += r
    valid += 1

print(f"{valid} valid readings, average {total / valid:.1f}")
```

```
4 valid readings, average 37.2
```

`r is None` asks "is `r` the empty value `None`?". For `None`, Python programmers write `is` instead of `==`.

⚡ A shortcut: list comprehensions

Building a new list with a loop and `.append()` is so common that Python has a one-line form for it, called a **list comprehension**. Read it as a sentence: "*make a list of* `x * 2` *for each* `x` *in* `nums`". Add an `if` at the end to keep only some items:

```
nums = [1, 2, 3, 4, 5]

doubled = [x * 2 for x in nums]
print(doubled)

# The long way gives the same result:
doubled_long = []
for x in nums:
    doubled_long.append(x * 2)
print(doubled_long)

hot = [t for t in temps_c if t >= 30]     # keep only the hot days
print(hot)
```

```
[2, 4, 6, 8, 10]
[2, 4, 6, 8, 10]
[33.0, 30.1]
```

No pressure: anything a list comprehension does, a normal loop can do too. Use whichever you find easier to read. You'll meet comprehensions a lot in other people's code.

⚠️ Common mistakes

- Starting the accumulator *inside* the loop (`total = 0` under the `for`): it's reset every round. Put it before the loop.
- Indenting a line that should run once after the loop (like the final `print`), so it runs every round.
- Forgetting the colon at the end of the `for` or `while` line.
- A `while` loop whose condition never becomes `False` → it's stopped after 20 seconds.
- `for i in 5:` → `TypeError`. Use `range(5)`.
- Forgetting that `range(1, 10)` stops at 9.

✅ Recap

- `for item in my_list:` runs the indented block once for each item.
- Accumulator: `total = 0` before the loop, `total += x` inside. Counter: `count += 1` inside an `if`.
- New list: `result = []` before, `result.append(...)` inside.
- `range(start, stop, step)` makes a series of numbers, stopping before `stop`.
- `enumerate()` gives index and item; `.items()` gives key and value.
- `while condition:` repeats as long as the condition is true.

- `break` leaves the loop; `continue` jumps to the next round.
- `[x * 2 for x in nums if x > 1]` builds a list in one line.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Total sales

Add up the week's sales with a `for` loop and store the result in `total`. Practise the accumulator pattern: **don't use `sum()`** this time.

```
sales = [230, 185, 310, 270, 405, 520, 390]
total = 0

# Loop over sales and add each amount to total
```

Hints:

1. Start the loop with `for amount in sales:` (don't forget the colon).
2. Inside the loop (4 spaces in), add the amount: `total += amount`

2. Count the hot days

Here are a (made-up) month's temperatures. Using a loop and a counter, count how many days were **30 degrees or more**, and store the answer in `hot_days`.

```
temps = [28, 31, 33, 29, 27, 30, 35, 36, 26, 24, 29, 31, 32, 30, 28,
         25, 27, 33, 34, 29, 30, 31, 26, 28, 30, 32, 29, 27, 31, 33]

# Count the days that are 30 or more
```

Hints:

1. Start a counter at zero before the loop: `hot_days = 0`
2. "30 or more" is `t >= 30`. Put an `if` inside the loop.
3. `for t in temps:` → `if t >= 30:` → `hot_days += 1`

3. A list of squares

Make a list called `squares` with the squares of the numbers 1 to 10: `[1, 4, 9, …, 100]`. Use `range()` (with a loop and `.append()`, or a list comprehension).

```
# Build the list of squares
```

Hints:

1. The numbers 1 to 10 are `range(1, 11)`, because range stops just before the stop number.
2. A square is `n ** 2` (or `n * n`).
3. `squares = []`, then `for n in range(1, 11):` → `squares.append(n ** 2)`

4. Countdown with while

Using a `while` loop, build a list called `countdown` that goes from `start` down to 1. With `start = 10` it should be `[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]`.

```
start = 10
countdown = []
number = start

# Fill countdown here, counting down from start
```

Hints:

1. Keep going as long as the number is above zero: `while number > 0:`
2. Inside the loop, append the number, then make it one smaller: `number -= 1`
3. If the page freezes for 20 seconds, you forgot `number -= 1`.

5. Challenge: over budget

Someone has a budget of 100 dollars for the week. The list shows what they spent each day (made up). Go through the days keeping a **running total**, and find the **index** of the first day on which the running total goes **over** the budget. Store it in `over_day`, and use `break` to stop as soon as you find it.

If the total never goes over the budget, `over_day` should stay `None`.

The checker will also try your code with other budgets, so use the variable `budget`.

```
spending = [23.5, 12.0, 31.2, 8.9, 27.4, 15.0, 19.3]
budget = 100
running_total = 0
over_day = None

# Find the index of the first day the running total goes over budget
```

Hints:

1. You need the index, so loop with `for i, amount in enumerate(spending):`
2. In each round, add the amount to `running_total`, then check `if running_total > budget:`
3. Inside that `if`: `over_day = i`, then `break`.

➡️ Tomorrow

Writing your own functions: you've been using functions like `len()`, `sum()` and `round()` all week. Tomorrow you'll make your *own*, with `def`. Wrap a piece of code (like today's average or Celsius-to-Fahrenheit loop) in a function once, give it a name, and reuse it anywhere.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Total sales

```
sales = [230, 185, 310, 270, 405, 520, 390]
total = 0

for amount in sales:
    total += amount
```

2. Count the hot days

```
temps = [28, 31, 33, 29, 27, 30, 35, 36, 26, 24, 29, 31, 32, 30, 28,
         25, 27, 33, 34, 29, 30, 31, 26, 28, 30, 32, 29, 27, 31, 33]

hot_days = 0
for t in temps:
    if t >= 30:
        hot_days += 1
```

3. A list of squares

```
squares = []
for n in range(1, 11):
    squares.append(n ** 2)
```

4. Countdown with while

```
start = 10
countdown = []
number = start

while number > 0:
    countdown.append(number)
    number -= 1
```

5. Challenge: over budget

```
spending = [23.5, 12.0, 31.2, 8.9, 27.4, 15.0, 19.3]
budget = 100
running_total = 0
over_day = None

for i, amount in enumerate(spending):
    running_total += amount
    if running_total > budget:
        over_day = i
        break
```